

2. Exercise: Parallel Sorting

Abstract

This study explores the implementation and performance evaluation of Parallel MergeSort and Parallel QuickSort, building upon their classical counterparts. The PRAM model, introduced for parallel computing, serves as the theoretical foundation. Classical MergeSort, known for its simplicity, and QuickSort, renowned for efficient partitioning, set the stage for parallel adaptations. The study culminates in an empirical analysis, testing the algorithms and providing insights into their practical implementation and efficiency.

1 Introduction

In the ever-evolving field of computer science, the rise of multi-core processors has brought parallel computing to the forefront. This study delves into the world of parallel algorithms, specifically exploring how two classic sorting methods—MergeSort and QuickSort—can be adapted and optimized. Grounded in the theoretical framework of the Parallel Random Access Machine (PRAM) model, designed for modern computing architectures, the study aims to enhance sorting algorithm performance to meet the demands of contemporary computational challenges. MergeSort and QuickSort, known for their distinctive methodologies, serve as the foundation for this exploration. The paper navigates the complexities of adapting these classic algorithms to parallel processing, addressing issues related to shared memory access and processor synchronization. Through empirical evaluation, the study aims to reveal the practical implications of these adaptations, offering insights into the efficiency and scalability of Parallel MergeSort and Parallel QuickSort. This study contributes to a deeper understanding of parallel sorting algorithms, guiding future algorithmic choices in

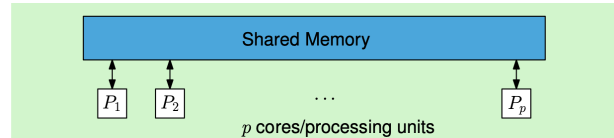


Figure 1: The PRAM model, From Storandt [2023]

real-world computational scenarios.

2 Preliminaries

2.1 PRAM Model

The classical RAM model assumes that the instructions are run sequentially one after another. That means that at every time step, exactly one instruction is executed.

However, a modern CPU has usually multiple cores that can execute multiple instructions in parallel. As this cannot be modeled with the classical RAM model, the PRAM model is used instead.

The PRAM model assumes a parallel machine with multiple processors p that can access a shared memory. Each of the processors can execute an instruction per timestep so that at every timestep multiple instructions are executed.

Because multiple instructions are executed at the same timestep, conflicts can occur between different processors if they access the same values in the shared memory. This leads to the need for strategies to decide, which processor is allowed to write at a specific position.

The three different possible strategies are:

- Common: all conflicting processors must want to write the same value, otherwise no value is written
- Arbitrary, an arbitrary writes the value

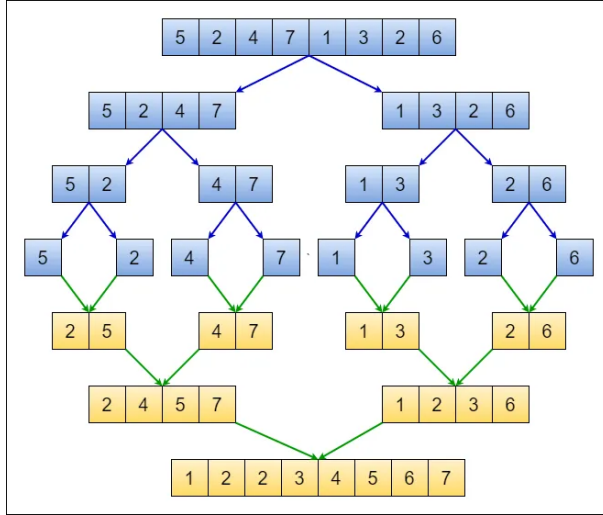


Figure 2: MergeSort Visualisation Vikram [2023]

- Priority: each processor is assigned a priority. the processor with the highest priority wins

2.2 Classical MergeSort

The MergeSort algorithm is a well-established sorting algorithm in the realm of computer science. It employs a divide-and-conquer strategy, recursively dividing the dataset into smaller chunks until individual elements are isolated. These elements are then gradually merged back together in a sorted order. In figure 2, an exemplary run of the MergeSort algorithm is visualised. The partitioning phase is colored in blue and the merging phase is colored in yellow.

MergeSort has an average time complexity of $\Theta(n \log n)$.

2.3 Classical QuickSort

The QuickSort algorithm is a widely-used sorting algorithm that belongs to the class of divide-and-conquer methods in computer science. It works by selecting a pivot element in the array and partitioning the other elements into two sub-arrays or segments according to whether they are less than or greater

Algorithm 1: MergeSort

Data: An array *arr* of length *n*

Result: Sorted array *arr*

```

1 if  $n > 1$  then
2    $mid \leftarrow n/2$ ;
3    $left \leftarrow arr[0 \dots mid - 1]$ ;
4    $right \leftarrow arr[mid \dots n - 1]$ ;
5   MergeSort(left);
6   MergeSort(right);
7   Merge(arr, left, right);

```

Algorithm 2: Merge

Data: An array *arr*, and two sorted arrays *left* and *right*

Result: Merged and sorted array *arr*

```

1  $i \leftarrow 0$ ;
2  $j \leftarrow 0$ ;
3  $k \leftarrow 0$ ;
4 while  $i < \text{length}(left)$  and  $j < \text{length}(right)$ 
5   do
6     if  $left[i] \leq right[j]$  then
7        $arr[k] \leftarrow left[i]$ ;
8        $i \leftarrow i + 1$ ;
9     else
10       $arr[k] \leftarrow right[j]$ ;
11       $j \leftarrow j + 1$ ;
12       $k \leftarrow k + 1$ ;
13 while  $i < \text{length}(left)$  do
14    $arr[k] \leftarrow left[i]$ ;
15    $i \leftarrow i + 1$ ;
16    $k \leftarrow k + 1$ ;
17 while  $j < \text{length}(right)$  do
18    $arr[k] \leftarrow right[j]$ ;
19    $j \leftarrow j + 1$ ;
20    $k \leftarrow k + 1$ ;

```

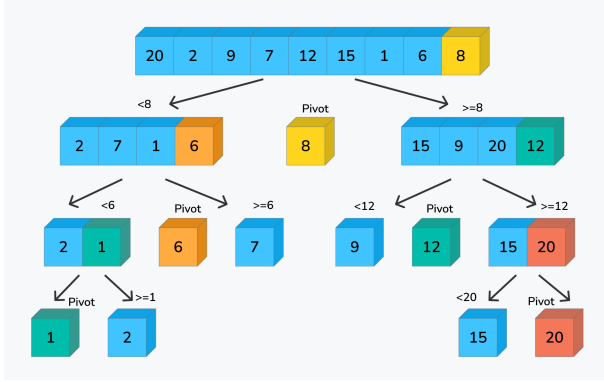


Figure 3: QuickSort Visualisation Chandak [2021]

than the pivot. For each sub-array, the process is repeated recursively.

QuickSort has an average time complexity of $\mathcal{O}(n \log n)$.

Algorithm 3: QuickSort

Data: An array *arr* of length *n*
Result: Sorted array *arr*

```

1 if n > 1 then
2   pivot ← SelectPivot(arr, n);
3   left ← {x ∈ arr | x < pivot};
4   right ← {x ∈ arr | x > pivot};
5   QuickSort(left);
6   QuickSort(right);
7   arr ← left + {pivot} + right;
```

Algorithm 4: SelectPivot

Data: An array *arr* and its length *n*
Result: Selected pivot element

```

1 return arr[n/2];
```

3 Algorithm & Implementation

3.1 Parallel MergeSort

In classical merge sort first divide the array (size = *n*) into 2 smaller arrays of size = $(1/2) * n$, each of which is divided into 2 arrays of size = $(1/4) * n$. This process continues until a single element is remaining, which we assume is sorted. The dividing part is not costly as there are only $\log(n)$ levels as the size of the array reduces by a factor of 2 in every round. The next phase of the mergesort algorithm involves merging of the 2 smaller sorted arrays to make a new sorted array. This step involves $\mathcal{O}(n)$ steps so paralyzing is crucial. To parallelize the merging of 2 sorted arrays the main problem we face is that the processors don't know the exact index in the target array. But this can be resolved by running binary search on the other array to find the number of elements below the element for every element. This step divides the task in *p* processors but also adds a $\log(n)$ factor for the binary search hence the total time complexity becomes: $(n/p) * (\log(n)^2)$.

3.2 Parallel QuickSort

In classical quick sort, we find a random pivot element and partition the array such that all elements smaller than the pivot are placed before it and all elements greater than the pivot are placed after it. Then 2 recursive calls are made to sort both the parts of the array. Here, the partitioning step takes $\mathcal{O}(n)$ time. which can be parallelized. We need to create partitions of size (n/p) , where *n*: input array size, *p*: number of processors. The problem in paralyzing this step is the different processors will not be in sync to know the location of elements in the target array. We can solve this problem by calculating elements smaller and larger than the pivot in every partition and storing them, and calculating a pre-sum array to know the exact amount of elements before a particular partition in $\mathcal{O}(1)$ time. But the pre-sum step requires $\mathcal{O}(p)$ time. Hence the time complexity becomes $(n/p) * \log(n) + p$.

Number of Processors	Running Time
1	0,39988
2	0,700455
3	0,835306
4	1,632089
5	2,178192
6	2,265259
7	2,892566
8	3,347607
9	4,351345
10	4,98114

Table 1: Running Time (in seconds) of Parallel MergeSort

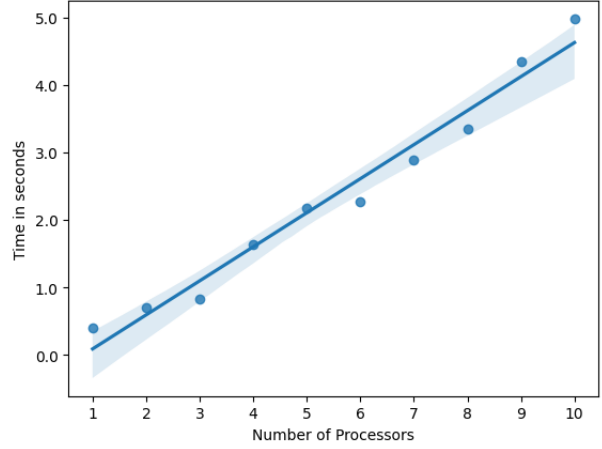


Figure 4: Parallel MergeSort - Varying Number of Processors

4 Experimental Evaluation

4.1 Data and Hardware

4.1.1 Dataset Generation

To evaluate the merge sorting algorithms, we create a vector of integers in each iteration that contains a particular number of integers n . The vector is generated with numbers $0..n$ which are then shuffled randomly. This way, we can make sure that the vector contains no duplicates. For the quick sorting algorithms vectors of the given input size were generated with random integers (duplication is allowed).

4.1.2 Hardware

The mergesort algorithm was evaluated on a machine with an Apple M2 CPU with 3.49 GHz and a memory of 32GB and the Quick sort algorithm was evaluated on a machine with an AMD Ryzen 5500u with 2.1 GHz and a memory of 24GB.

4.2 Results

4.2.1 Parallel MergeSort

We evaluated the parallel MergeSort by using it to sort a list of numbers with a size of $10^6 = 1000000$ with a varying number of processors to use. The results are displayed in table 1 and plotted in figure 4.

Input Size	Running Time
10^2	0,000258
10^3	0,001861
10^4	0,020036
10^5	0,165674
10^6	1,743887
10^7	16,700938
10^8	183,595084
10^9	1731,937798

Table 2: Running Time (in seconds) of Parallel MergeSort using 4 cores

Additionally, we used a configuration with 4 processors to evaluate lists with a size from 10^2 to 10^9 elements. The results of this experiment are displayed in table 2 and plotted in figure 5 and 6

4.2.2 QuickSort

For the quicksort algorithm the run time for $n = 1000000$ were: Also, the run time for sequential quicksort was 10.909 seconds. Here we see that the best running time come when algorithm is run with $p = 1$. Hence, running the quick sort for $p = 1$ with $n = 10^i$ ($i = 1, 2, \dots, 7$).

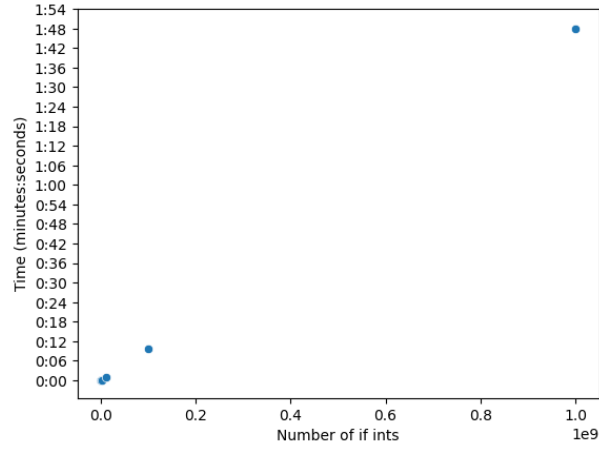


Figure 5: Parallel MergeSort - Varying Input Size with 4 Processors

Number of Cores	Total Time (in seconds)
1	17.477
2	119.086
3	119.902
4	123.259
5	132.484
6	143.240
7	164.228
8	174.826

Table 3: Performance Metrics for Different Numbers of Cores for QuickSort

Number of input elements	Total Time (in seconds)
10	0
100	0.001
1000	0.013
10000	0.135
100000	1.386
1000000	19.085
10000000	218.349

Table 4: Performance Metrics for Different Numbers of Cores for QuickSort

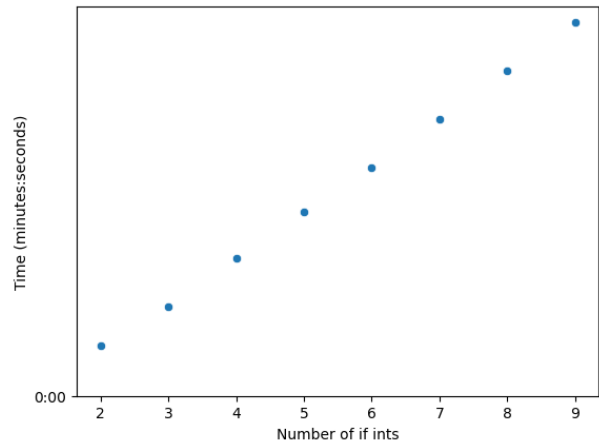


Figure 6: Log version of 5

5 Discussion and Conclusion

In quick sort, we can see the sequential quick sort and the parallel quick sort have comparable time, which is as per expectation. But we see that the time increases as we increase the number of cores in the algorithm, this might be a result of multi-core usage in the single-core and sequential algorithm itself, which might be better optimized by the operating system and the processor.

In MergeSort, we can see a very similar result. With a higher number of cores, the execution gets slower while utilising more CPU resources.

References

G. Chandak. Sorting algorithms (quick sort, merge sort). <https://workat.tech/problem-solving/>

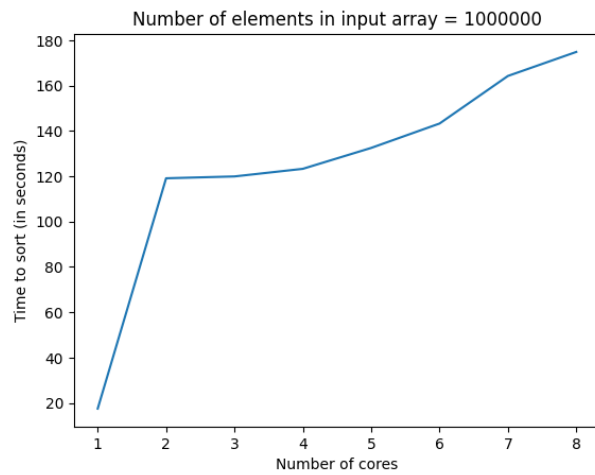


Figure 7: QuickSort run time analysis.

tutorial/sorting-algorithms-quick-sort-merge-sort-dsa-tutorials-6j3h98lk6j2w, 2021.

S. Storandt. Lecture: 2. memory-aware algorithms, 2023.

G. Vikram. Visualizing, designing, and analyzing the merge sort algorithm. <https://medium.com/javarevisited/visualizing-designing-and-analyzing-the-merge-sort-algorithm-904ceb78a592>, 2023.